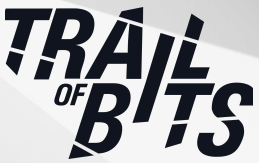

Offchain Labs Yield-Bearing Bridge

Security Assessment (Summary Report)

August 7, 2026

Prepared for:
Harry Kalodner, Steven Goldfeder, and Ed Felten
Offchain Labs

Prepared by: **Simone Monica and Jaime Iglesias**

Table of Contents

Project Summary

Contact Information

The following project manager was associated with this project:

Mary O'Brien, Project Manager
mary.obrien@trailofbits.com

The following engineering director was associated with this project:

Benjamin Samuels, Engineering Director, Blockchain
benjamin.samuels@trailofbits.com

The following consultants were associated with this project:

Simone Monica, Consultant
simone.monica@trailofbits.com

Jaime Iglesias, Consultant
jaime.iglesias@trailofbits.com

Project Timeline

The significant events and milestones of the project are listed below.

Date	Event
May 11, 2026	Pre-project kickoff call
June 8, 2026	Delivery of report draft
July 15, 2026	Completion of fix review
August 7, 2026	Delivery of final summary report

Project Targets

The engagement involved reviewing and testing the target listed below.

token-bridge-contracts

Repository	https://github.com/OffchainLabs/token-bridge-contracts
Version	cdbd23820e0d746e1b505cc8a4463d282a5f36d3 6813a97a66c54c16a3d1a5692b45569ffae5fa22
Type	Solidity
Platform	Arbitrum

Executive Summary

Engagement Overview

Offchain Labs engaged Trail of Bits to review a new yield-bearing bridge feature for the ERC-20 native token bridge, allowing Arbitrum chain owners to generate yield on escrowed tokens.

The yield-bearing bridge feature is not opinionated about how yield is generated; it simply provides a master vault (based on ERC-4626) through which the owner can manage generated yield. Note that yield generation is not opt-in; once an owner deploys the yield-bearing bridge, all deposits will flow through the master vault. Finally, the feature is constrained to ERC-20 and custom gateways and inherits the same limitations as the original protocol (e.g., no custom tokens such as ERC-777 or fee-on-transfer tokens).

A team of two consultants conducted the review from May 11 to May 22, 2026, for a total of four engineer-weeks of effort. With full access to source code and documentation, we performed static and dynamic testing of the targets, using automated and manual processes.

Observations and Impact

The main focus of the review was to determine whether this new feature is correctly implemented. We looked for potential logical issues, issues related to Arbitrum-specific behavior, and unintended consequences arising from the changes introduced.

Recommendations

- **Remediate the findings disclosed in this report.** These findings should be addressed through direct fixes or broader refactoring efforts.

Summary of Findings

The table below summarizes the findings of the review, including details on type and severity.

ID	Title	Type	Severity
1	Dead code in the _outboundTransferCustomRefund function	Undefined Behavior	**Informational**
2	Undefined behavior when the vault has a loss	Undefined Behavior	**Informational**
3	Orbit yield-bearing bridge gateway breaks previous assumptions	Undefined Behavior	**High**
4	Incomplete supportsInterface function	Undefined Behavior	**Informational**
5	Profit distribution can fail unexpectedly	Denial of Service	**Informational**

Detailed Findings

1. Dead code in the _outboundTransferCustomRefund function

Severity: **Informational**	Difficulty: **Low**
Type: Undefined Behavior	Finding ID: TOB-ARBYIELD-1
Target: `tokenbridge/ethereum/gateway/L1ArbitrumGateway.sol`	

Description

The `_outboundTransferCustomRefund` function, which contains the core logic for L1-to-L2 deposits, contains dead code.

As shown in figure 1.1, one of the main constraints the function imposes is that the caller must be the router contract, thereby rejecting direct calls; see the highlighted `require` statement.

```
function _outboundTransferCustomRefund(
    address _l1Token,
    address _refundTo,
    address _to,
    uint256 _amount,
    uint256 _maxGas,
    uint256 _gasPriceBid,
    bytes calldata _data
) internal virtual returns (bytes memory res, uint256 amountOnL2) {
    require(isRouter(msg.sender), "NOT_FROM_ROUTER");
    // This function is set as public and virtual so that subclasses can override
    // it and add custom validation for callers (ie only whitelisted users)
    require(_l1Token.isContract(), "L1_NOT_CONTRACT");
    {
        address l2Token = calculateL2TokenAddress(_l1Token);
        require(l2Token != address(0), "NO_L2_TOKEN_SET");
    }
    uint256 seqNum;
    {
        address _from;
        {
            bytes memory extraData;
            uint256 _maxSubmissionCost;
            uint256 tokenTotalFeeAmount;
            if (super.isRouter(msg.sender)) {
                // router encoded
                (_from, extraData) =
```

```
GatewayMessageHandler.parseFromRouterToGateway(_data);
            } else {
                _from = msg.sender;
                extraData = _data;
            }
```

Figure 1.1: Part of the `_outboundTransferCustomRefund` function in L1ArbitrumGateway.sol#L259–L289

However, the `if/else` statement suggests that the function is expected to be callable from a different address than the router, which would cause the `_from` and `_data` fields to be parsed differently. But because of the first `require` statement, the `else` branch is dead code.

Recommendations

Short term, determine whether the dead code should be removed or whether the logic needs to be changed to support the `else` statement, and update the function accordingly.

Long term, thoroughly document the expected behavior of each function and include tests to verify it.

2. Undefined behavior when the vault has a loss	
Severity: **Informational**	Difficulty: **Low**
Type: Undefined Behavior	Finding ID: TOB-ARBYIELD-2
Target: `tokenbridge/libraries/vault/MasterVault.sol`	

Description

When the vault has losses, the share calculation uses a proportional formula to socialize losses among depositors; otherwise, the calculation uses a 1:1 ratio of shares for deposited assets.

However, new depositors are not prevented from depositing into the vault after it has already incurred losses. When the vault has a loss, each share will be worth less than one unit of the underlying asset, so new depositors could deposit at a discounted rate and make a profit if the vault is made whole before they withdraw.

```
function _convertToSharesRoundDown(uint256 assets) internal view returns (uint256
shares) {
        // bias against the depositor by rounding DOWN totalAssets to more easily
detect losses
        if (_haveLoss()) {
            // we have losses
            return assets.mulDiv(
                totalSupply(),
                _totalAssets(MathUpgradeable.Rounding.Up),
                MathUpgradeable.Rounding.Down
            );
        }
        // no losses, use ideal 1:1 ratio
        return assets;
    }
```

Figure 2.1: The `_convertToSharesRoundDown` function in `MasterVault.sol#L517–L529`

Recommendations

Short term, determine whether this is the intended behavior of the bridge. If not, then consider implementing mechanisms to prevent this behavior, such as rejecting new deposits while the vault is in a loss state or implementing pre-deposit NAVs; note that such NAVs would add complexity to the bridge.

Long term, thoroughly document the bridge's behavior when the vault is in a loss state so that users are aware of it.

3. Orbit yield-bearing bridge gateway breaks previous assumptions

Severity: **High**	Difficulty: **Low**
Type: Undefined Behavior	Finding ID: TOB-ARBYIELD-3
Target: `contracts/tokenbridge/ethereum/gateway/L1OrbitERC20Gateway.sol`	

Description

Because Orbit chains have their own native tokens, the Orbit bridge has two requirements that other bridge types do not; these requirements are enforced in the `outboundTransferCustomRefund` function, as shown in figure 3.1:

- `msg.value` must be zero, since fees are paid in the Orbit chain's native token rather than in ETH.
- The asset being bridged cannot be the native token, as it would create a representation on the child chain separate from the official one.

```
function outboundTransferCustomRefund(
    address _l1Token,
    address _refundTo,
    address _to,
    uint256 _amount,
    uint256 _maxGas,
    uint256 _gasPriceBid,
    bytes calldata _data
) public payable override returns (bytes memory res) {
    // fees are paid in native token, so there is no use for ether
    require(msg.value == 0, "NO_VALUE");

    // We don't allow bridging of native token to avoid having multiple
representations of it
    // on child chain. Native token can be bridged directly through inbox using
depositERC20().
    require(_l1Token != _getNativeFeeToken(), "NOT_ALLOWED_TO_BRIDGE_FEE_TOKEN");
```

Figure 3.1: The `outboundTransferCustomRefund` function in L1OrbitERC20Gateway.sol#L19–L33

However, as shown in figure 3.2, the yield-bearing bridge gateway includes a new function `outboundTransferCustomRefundWithSlippageTolerance`, which does not include these two checks.

```
function outboundTransferCustomRefundWithSlippageTolerance(
    address _l1Token,
    address _refundTo,
    address _to,
    uint256 _amount,
    uint256 _maxGas,
    uint256 _gasPriceBid,
    bytes calldata _data,
    uint256 minReceivedOnL2
) public payable absYbbNonReentrant returns (bytes memory res) {
    uint256 receivedOnL2;
    (res, receivedOnL2) = _outboundTransferCustomRefund(
        _l1Token,
        _refundTo,
        _to,
        _amount,
        _maxGas,
        _gasPriceBid,
        _data
    );
    require(receivedOnL2 >= minReceivedOnL2, "SLIPPAGE_EXCEEDED");
}
```

Figure 3.2: The `outboundTransferCustomRefundWithSlippageTolerance` function in AbsYbbGateway.sol#L42–L63

This function breaks two main assumptions made by the Orbit bridge, which can cause ETH to be trapped in the bridge (since calls that include a nonzero `msg.value` are no longer rejected) and the native token to have multiple representations on the child chain.

Exploit Scenario

Eve uses the new yield-bearing bridge gateway to bridge a native asset of an Orbit chain, but she accidentally sends ETH as part of the transaction. As a result, her ETH is permanently trapped in the bridge without the possibility of recovery.

Recommendations

Short term, in the `outboundTransferCustomRefundWithSlippageTolerance` function, add the same checks that are performed in the `outboundTransferCustomRefund` function.

Long term, when adding new functionality that extends existing functionality, make sure to include new tests to verify that the previous invariants are still maintained.

4. Incomplete supportsInterface function

Severity: **Informational**	Difficulty: **Low**
Type: Undefined Behavior	Finding ID: TOB-ARBYIELD-4
Target: `contracts/tokenbridge/ethereum/gateway/L1ArbitrumGateway.sol`	

Description

The `supportsInterface` function in the `L1ArbitrumGateway` contract is missing the new `outboundTransferCustomRefundWithSlippageTolerance` function. Not including this function breaks ERC-165 compliance.

```
function supportsInterface(bytes4 interfaceId)
    public
    view
    override(ERC165, IERC165)
    returns (bool)
{
    // registering interfaces that is added after arb-bridge-peripherals >1.0.11
    // using function selector instead of single function interfaces to reduce bloat
    return
        interfaceId == this.outboundTransferCustomRefund.selector ||
        super.supportsInterface(interfaceId);
}
```

Figure 4.1: The `supportsInterface` function in `L1ArbitrumGateway.sol#L356–L368`

Recommendations

Short term, include the `outboundTransferCustomRefundWithSlippageTolerance` function in `supportsInterface`.

Long term, when adding new functionality for contracts that support ERC-165, make sure to properly update their interfaces.

5. Profit distribution can fail unexpectedly

Severity: **Informational**	Difficulty: **Medium**
Type: Denial of Service	Finding ID: TOB-ARBYIELD-5
Target: `contracts/tokenbridge/libraries/vault/MasterVault.sol`	

Description

The `_distributePerformanceFee` function unconditionally withdraws the full profit amount from the subvault without checking whether the subvault can satisfy the withdrawal. If the subvault holds insufficient liquid assets, the call reverts and fee distribution halts entirely until the subvault regains liquidity.

When idle assets held directly by the `MasterVault` contract fall short of the accrued profit, the function computes `amountToWithdraw` as `profit - amountToTransfer` and calls `subVault.withdraw` with that full amount, as shown in figure 5.1.

The ERC-4626 withdrawal specification requires the call to revert when the requested amount exceeds `maxWithdraw`. Because `_distributePerformanceFee` does not cap `amountToWithdraw` against `subVault.maxWithdraw(address(this))`, any temporary illiquidity in the subvault causes the entire distribution to fail. The accrued fee is preserved in state and will be redistributed once liquidity recovers, but no partial distribution is possible.

```
uint256 amountToWithdraw = profit - amountToTransfer;

if (amountToTransfer > 0) {
    asset.safeTransfer(beneficiary, amountToTransfer);
}
if (amountToWithdraw > 0) {
    // slither-disable-next-line unused-return
    subVault.withdraw(amountToWithdraw, beneficiary, address(this));
}
```

Figure 5.1: Part of the `_distributePerformanceFee` function in MasterVault.sol#L494–L503

By contrast, `_rebalanceToTarget` already caps withdrawals against `maxWithdraw`, demonstrating awareness of this constraint elsewhere in the contract.

```
function _rebalanceToTarget(int256 minExchRateWad) private {
    [...]
```

```
    if (idleBalance < idleTargetDown) {
        uint256 desiredWithdraw = idleTargetDown - idleBalance;
        uint256 maxWithdrawable = subVault.maxWithdraw(address(this));
        uint256 withdrawAmount =
            desiredWithdraw < maxWithdrawable ? desiredWithdraw : maxWithdrawable;
```

Figure 5.2: Part of the `_rebalanceToTarget` function in `MasterVault.sol#L305–L308`

Exploit Scenario

The `MasterVault` contract accumulates profit over time. When a keeper calls a function that triggers `_distributePerformanceFee`, the idle balance covers only part of the accrued profit, so the function attempts to withdraw the remainder from the subvault.

The subvault, however, has its assets locked in a position with reduced liquidity—`maxWithdraw` returns a value smaller than `amountToWithdraw`. The `subVault.withdraw` call reverts, causing the entire transaction to revert.

The beneficiary receives no fees. Every subsequent distribution attempt fails in the same way until the subvault's liquidity recovers, delaying fee payments indefinitely.

Recommendations

Short term, update `_distributePerformanceFee` so that `amountToWithdraw` is capped at `subVault.maxWithdraw(address(this))` before the call to `subVault.withdraw`, and have it emit the `PerformanceFeesWithdrawn` event with the actual amounts transferred. This will allow partial distribution to succeed immediately and defer the remainder to future calls when subvault liquidity has recovered.

Long term, document the invariant that fee distribution may be partial, and ensure monitoring or alerting is in place so that operators can detect accumulated, undistributed fees caused by subvault illiquidity. This will prevent silent liveness degradation from going unnoticed over extended periods.

A. Vulnerability Categories

The following tables describe the vulnerability categories, severity levels, and difficulty levels used in this document.

Vulnerability Categories	
Category	**Description**
Access Controls	Insufficient authorization or assessment of rights
Auditing and Logging	Insufficient auditing of actions or logging of problems
Authentication	Improper identification of users
Configuration	Misconfigured servers, devices, or software components
Cryptography	A breach of system confidentiality or integrity
Data Exposure	Exposure of sensitive information
Data Validation	Improper reliance on the structure or values of data
Denial of Service	A system failure with an availability impact
Error Reporting	Insecure or insufficient reporting of error conditions
Patching	Use of an outdated software package or library
Session Management	Improper identification of authenticated users
Testing	Insufficient test methodology or test coverage
Timing	Race conditions or other order-of-operations flaws
Undefined Behavior	Undefined behavior triggered within the system

Severity Levels	
Severity	**Description**
Informational	The issue does not pose an immediate risk but is relevant to security best practices.
Undetermined	The extent of the risk was not determined during this engagement.
Low	The risk is small or is not one the client has indicated is important.
Medium	User information is at risk; exploitation could pose reputational, legal, or moderate financial risks.
High	The flaw could affect numerous users and have serious reputational, legal, or financial implications.

Difficulty Levels	
Difficulty	**Description**
Not Applicable	This issue is of informational severity and does not pose an immediate risk, so difficulty does not apply.
Undetermined	The difficulty of exploitation was not determined during this engagement.
Low	The flaw is well known; public tools for its exploitation exist or can be scripted.
Medium	An attacker must write an exploit or will need in-depth knowledge of the system.
High	An attacker must have privileged access to the system, may need to know complex technical details, or must discover other weaknesses to exploit this issue.

B. Code Quality Findings

The following findings are not associated with any specific vulnerabilities. However, fixing them will enhance code readability and may prevent the introduction of vulnerabilities in the future.

- It is possible to call `_setupdecimals` in `initialize` instead of overriding the `decimals` function.

```
/// @notice Returns the decimals of the underlying asset
/// @dev    Requires underlying asset to implement IERC20Metadata.decimals()
function decimals() public view override returns (uint8) {
    return IERC20Metadata(address(asset)).decimals();
}
```

Figure B.1: The `decimals` function in MasterVault.sol#L453–L455

- The operation described in the highlighted comment in figure B.2 no longer exists; fees are always on.

```
/// @notice The fee manager can:
///         - Toggle performance fees on/off
///         - Set the performance fee beneficiary
bytes32 public constant FEE_MANAGER_ROLE = keccak256("FEE_MANAGER_ROLE");
```

Figure B.2: Comment about nonexistent toggle in MasterVaultRoles.sol#L24–L27

- The balance check shown in figure B.3 can be triggered by sending tokens to the vault during the migration. Consider replacing the current two-step approach (draining the current vault then setting the new one) to an atomic approach that drains the current vault and sets the new vault simultaneously.

```
function setSubVault(IERC4626 _subVault) external nonReentrant
onlyRole(GENERAL_MANAGER_ROLE) {
    if (!isSubVaultWhitelisted(address(_subVault))) {
        revert SubVaultNotWhitelisted(address(_subVault));
    }
    if (address(_subVault.asset()) != address(asset)) revert
SubVaultAssetMismatch();

    // we ensure target allocation is zero, therefore the master vault holds no
subvault shares
    if (targetAllocationWad != 0) revert
NonZeroTargetAllocation(targetAllocationWad);

    // sanity check to ensure we have zero subvault shares before changing
    if (subVault.balanceOf(address(this)) != 0) {
        revert NonZeroSubVaultShares(subVault.balanceOf(address(this)));
    }
```

Figure B.3: The `setSubVault` function in `MasterVault.sol#L349–L361`

C. Fix Review Results

When undertaking a fix review, Trail of Bits reviews the fixes implemented for issues identified in the original report. This work involves a review of specific areas of the source code and system configuration, not a comprehensive analysis of the system.

On July 15, 2026, Trail of Bits reviewed the fixes and mitigations implemented by the Offchain Labs team for the issues identified in this report. We reviewed each fix to determine its effectiveness in resolving the associated issue.

In summary, Offchain Labs has resolved all five issues described in this report.

For additional information, please see the Detailed Fix Review Results below.

ID	Title	Severity	Status
1	Dead code in the _outboundTransferCustomRefund function	Informational	Resolved
2	Undefined behavior when the vault has a loss	Informational	Risk Accepted
3	Orbit yield-bearing gateway breaks previous assumptions	High	Resolved
4	Incomplete supportsInterface function	Informational	Resolved
5	Profit distribution can fail unexpectedly	Informational	Resolved

Detailed Fix Review Results

TOB-ARBYIELD-1: Dead code in the _outboundTransferCustomRefund function
Resolved. The dead code has been removed.

TOB-ARBYIELD-2: Undefined behavior when the vault has a loss
Risk accepted. The issue was acknowledged by the client.

TOB-ARBYIELD-3: Orbit yield-bearing bridge gateway breaks previous assumptions
Resolved. The `_outboundTransferCustomRefund` function now explicitly rejects ETH and prevents the native fee token from being bridged.

TOB-ARBYIELD-4: Incomplete supportsInterface function
Resolved. The `supportsInterface` function has been updated or added in the `AbsYbbGateway`, `L1OrbitYbbCustomGateway`, `L1YbbCustomGateway`, and `L1YbbERC20Gateway` contracts.

TOB-ARBYIELD-5: Profit distribution can fail unexpectedly
Resolved. Temporary illiquidity now triggers a partial distribution rather than causing a revert.

D. Fix Review Status Categories

The following table describes the statuses used to indicate whether an issue has been sufficiently addressed.

Fix Status	
Status	**Description**
Undetermined	The status of the issue was not determined during this engagement.
Risk Accepted	The issue was acknowledged by the client.
Unresolved	The issue persists and has not been resolved.
Partially Resolved	The issue persists but has been partially resolved.
Resolved	The issue has been sufficiently resolved.

About Trail of Bits

Founded in 2012 and headquartered in New York, Trail of Bits provides technical security assessment and advisory services to some of the world's most targeted organizations. We combine high-end security research with a real-world attacker mentality to reduce risk and fortify code. With 100+ employees around the globe, we've helped secure critical software elements that support billions of end users, including Kubernetes and the Linux kernel.

We maintain an exhaustive list of publications at https://github.com/trailofbits/publications, with links to papers, presentations, public audit reports, and podcast appearances.

In recent years, Trail of Bits consultants have showcased cutting-edge research through presentations at CanSecWest, HCSS, Devcon, Empire Hacking, GrrCon, LangSec, NorthSec, the O'Reilly Security Conference, PyCon, REcon, Security BSides, and SummerCon.

We specialize in software testing and code review assessments, supporting client organizations in the technology, defense, blockchain, and finance industries, as well as government entities. Notable clients include HashiCorp, Google, Microsoft, Western Digital, Uniswap, Solana, Ethereum Foundation, Linux Foundation, and Zoom.

To keep up with our latest news and announcements, please follow @trailofbits on X or LinkedIn and explore our public repositories at https://github.com/trailofbits. To engage us directly, visit our "Contact" page at https://www.trailofbits.com/contact or email us at info@trailofbits.com.

Trail of Bits, Inc.
228 Park Ave S #80688
New York, NY 10003
https://www.trailofbits.com
info@trailofbits.com

Notices and Remarks

Copyright and Distribution

Test Coverage Disclaimer

Trail of Bits performed all activities associated with this project in accordance with a statement of work and an agreed-upon project plan.

Security assessment projects are time-boxed and often rely on information provided by a client, its affiliates, or its partners. As a result, the findings documented in this report should not be considered a comprehensive list of security issues, flaws, or defects in the target system or codebase.

Trail of Bits uses automated testing techniques to rapidly test software controls and security properties. These techniques augment our manual security review work, but each has its limitations. For example, a tool may not generate a random edge case that violates a property or may not fully complete its analysis during the allotted time. A project's time and resource constraints also limit their use.